

Name: Atchaya Vharsne S

Reg. No.: 1922524185

Course Code: CSA0401

Course Name: Operating Systems

CO2 AT2 SCENARIO BASED

Question 1 — CPU Scheduling in a Real-Time Railway Ticket Booking System

I. Most Suitable Scheduling Algorithm

Priority Scheduling is the most suitable algorithm for this scenario.

Priority Assignment:

- P1 – Payment Gateway Transactions → High Priority
- P2 – Seat Availability Checks → Medium Priority
- P3 – Ticket Confirmation and Printing → Low Priority

Since payment processing is the most critical task, it should always receive CPU time first to avoid transaction failures.

II. Justification

Priority Scheduling improves system performance because:

- Critical tasks are executed first, ensuring payment transactions are completed quickly.
- CPU utilization is improved since urgent processes are not kept waiting behind less important tasks.
- Customer waiting time decreases because payments and seat verification are completed faster, allowing ticket confirmation to proceed sooner.
- It is suitable for real-time systems where certain tasks are more important than others.

III. Sample Execution Order

Suppose the following processes arrive at the same time:

Process	Task	Priority	Burst Time
P1	Payment Gateway	High	5 ms
P2	Seat Availability	Medium	8 ms
P3	Ticket Confirmation & Printing	Low	10 ms

Execution Order:

P1 → P2 → P3

Gantt Chart:



- P1 finishes at 5 ms
- P2 finishes at 13 ms
- P3 finishes at 23 ms

IV. One Drawback

The main drawback of Priority Scheduling is starvation.

- Low-priority processes (such as P3 – Ticket Confirmation) may wait for a long time if high-priority tasks keep arriving continuously.
- During festival seasons with many payment requests, ticket confirmation may be delayed indefinitely.

Solution: Use Aging, where the priority of waiting processes gradually increases so that every process eventually gets CPU time.

Question 2 — Deadlock in a Banking Transaction Server

I. Text-Based Resource Allocation Graph (RAG)

Resources:

A = Account Database

B = Customer Info Database

C = Transaction Logs

T1 holds A and requests B

A $\longrightarrow$ T1 $\longrightarrow$ B

T2 holds B and requests C

B $\longrightarrow$ T2 $\longrightarrow$ C

T3 holds C and requests A

C $\longrightarrow$ T3 $\longrightarrow$ A

Or as a cycle:

A $\rightarrow$ T1 $\rightarrow$ B $\rightarrow$ T2 $\rightarrow$ C $\rightarrow$ T3 $\rightarrow$ A

II. Has a Deadlock Occurred? Justify.

Yes, a deadlock has occurred.

Justification: The system forms a **circular wait**:

- T1 holds A and waits for B.
- T2 holds B and waits for C.
- T3 holds C and waits for A.

Since each transaction is waiting for a resource held by another transaction, none of them can continue. Therefore, all four Coffman conditions are satisfied:

- Mutual Exclusion
- Hold and Wait
- No Pre-emption
- Circular Wait

Hence, the system is in a deadlock state.

III. Recommended Technique

Deadlock Prevention (Resource Ordering)

Assign a fixed order for acquiring resources:

$A \rightarrow B \rightarrow C$

Every transaction must request resources only in this order.

For example:

- **T1:** Lock A $\rightarrow$ Lock B
- **T2:** Lock B only after acquiring A (if A is needed), then C
- **T3:** Lock A first, then C (or A $\rightarrow$ B $\rightarrow$ C if all are required)

How it fixes the issue

A fixed resource ordering eliminates circular wait, so a deadlock cannot occur because no transaction can request a lower-ordered resource after holding a higher-ordered one.

IV. Suggested Redesign for Locking Order

Use the following global locking order for all transactions:

Step 1: Lock A (Account Database)

Step 2: Lock B (Customer Info Database)

Step 3: Lock C (Transaction Logs)

Perform transaction

Release locks in reverse order:

Unlock C $\rightarrow$ Unlock B $\rightarrow$ Unlock A

Example

- T1: A $\rightarrow$ B
- T2: A $\rightarrow$ B $\rightarrow$ C
- T3: A $\rightarrow$ C

Since every transaction follows the same lock acquisition order, circular waiting is prevented, avoiding future deadlocks.

Question 3 — Mobile OS App Switching Lag

I. Why does the CPU remain idle despite many active processes?

The CPU can remain idle because:

- Many background apps are waiting for I/O operations (network, storage, user input) rather than needing CPU time.
- When no process in the Ready Queue is available, the CPU stays idle.
- Frequent context switching among 20 background apps also wastes CPU time, reducing effective CPU usage.

II. Is the 20 ms time quantum helping or hurting performance?

The 20 ms time quantum is hurting performance.

Reason:

- With 20 active background apps, the CPU switches between processes very frequently.
- Frequent context switching increases overhead and wastes CPU time.
- Interactive apps like WhatsApp and the music player experience delays because the CPU spends more time switching than executing useful work.

III. Recommendation

Recommended Scheduling Method: Multilevel Feedback Queue (MLFQ)

OR

Increase the Round Robin time quantum from 20 ms to about 50 ms.

Justification:

- MLFQ gives higher priority to interactive (foreground) apps such as WhatsApp.
- Background apps receive lower priority but still get CPU time.
- If continuing with Round Robin, a 50 ms quantum reduces unnecessary context switches while maintaining good responsiveness.

IV. How will this reduce lag and improve throughput?

- Foreground apps (WhatsApp, social media) receive CPU time more quickly, making app switching smoother.
- The music player continues running with fewer interruptions.
- Fewer context switches reduce CPU overhead.
- More CPU time is spent executing processes instead of switching between them.
- This improves system throughput, reduces switching lag, and provides a better user experience.

Question 4 — Deadlock Risk in an Operating System Update

I. Can this sequence lead to a deadlock?

Yes, this sequence can lead to a deadlock.

Reason:

- M1 holds R1 and waits for R2.
- M2 holds R2 and waits for R3.
- M3 holds R3 and waits for R1.

Each module is waiting for a resource held by another module, so none of them can continue.

II. Exact Circular Wait Condition

The circular wait is:

M1 holds R1 → waits for R2

M2 holds R2 → waits for R3

M3 holds R3 → waits for R1

Or simply:

R1 → M1 → R2 → M2 → R3 → M3 → R1

This forms a cycle, satisfying the circular wait condition, which is one of the four necessary conditions for deadlock.

III. Banker's Algorithm Style Resource Allocation

Use Banker's Algorithm to allocate resources only if the system remains in a safe state.

Example:

Before granting a resource request:

- Check whether enough resources are available.
- Simulate the allocation.
- If all modules can still complete in some order, grant the request.
- Otherwise, delay the request until resources become available.

Safe execution sequence (example):

M1 → M2 → M3 → M4

or any other sequence that keeps the system in a safe state.

This prevents the system from entering an unsafe state where deadlock could occur.

IV. How OS Designers Can Prioritize Modules

Use Priority Scheduling during the update.

Suggested priorities:

Module	Task	Priority
M1	Copy update files	High
M2	Install drivers	High
M3	Update registry	Medium
M4	Validate system integrity	Low

Benefits

- Critical update tasks complete first.
- Resources are released sooner for other modules.
- CPU spends less time waiting for locked resources.
- Overall CPU utilization and update performance improve.

Question 5 — Cloud Server CPU Utilization Analysis

The CPU utilization formula is:

$$\text{CPU Utilization} = 1 - p^n$$

where:

- p = Probability that a VM is waiting for I/O
- n = Number of VMs

Given:

- $p = 0.8$ (80% I/O wait)
- $n = 7$ VMs

I. Calculate CPU Utilization

$$\begin{aligned}\text{CPU Utilization} &= 1 - (0.8)^7 \\ &= 1 - 0.2097152 \\ &= 0.7902848\end{aligned}$$

$$\text{CPU Utilization} \approx 79.03\%$$

II. Interpretation

The server is reasonably well utilized.

- CPU utilization is approximately 79%.
- This means the CPU is busy most of the time and idle only about 21% of the time.
- Therefore, the server is optimally loaded, though there is still some room to improve utilization.

III. Effect of Changing the Number of VMs

- **Increasing the number of VMs:**
 - CPU utilization generally increases because while some VMs wait for I/O, others can use the CPU.
 - Throughput improves up to an optimal point.
 - Too many VMs can eventually cause excessive context switching and resource contention, reducing performance.

- **Decreasing the number of VMs:**
 - CPU utilization decreases because fewer VMs are available to execute when others are waiting for I/O.
 - Overall throughput also decreases.

IV. Two OS-Level Strategies to Improve CPU Utilization

Strategy 1: Multiprogramming

- Allow more ready processes or VMs to execute while others are waiting for network or disk I/O.
- This keeps the CPU busy and increases utilization.

Strategy 2: Efficient CPU Scheduling

- Use scheduling algorithms such as Multilevel Feedback Queue (MLFQ) or Round Robin with an appropriate time quantum.
- These algorithms reduce CPU idle time, improve responsiveness, and increase overall throughput.